Project guide

GitHub Graveyard

Walk through a 3D cemetery of any GitHub user's forgotten repos

Enter a username. Each public repository becomes a grave, shaped by its stars and weathered by how long it has gone without a push.

Made by	Version	Platform	Built with
Gopinath	**0.1.0 MVP**	**Web browser**	**Next.js 16, Three.js**

© 2026 Gopinath. All rights reserved.

Updated 31 Oct 2026, github.com/goprocker/GitHubGraveyard

© 2026 Gopinath. All rights reserved.

Contents

Inside this guide

01 What it is and why it exists

02 Every feature

03 How it looks

04 Tech stack

05 Architecture

06 How each part works

07 Key decisions

08 Repository map

09 Install, run and deploy

10 Testing and verified results

11 Limits and known gaps

12 Learn the project: words and data flow

13 Hard problems and what they taught

14 Interview questions and cheat sheet

© 2026 Gopinath. All rights reserved.

Section 01

What it is and why it exists

Every developer has a trail of abandoned side projects. A profile page shows them as a flat list. GitHub Graveyard turns that list into a place you can walk through: a moonlit cemetery where the state of each repository is visible at a glance.

It is a playful project, but it needs real engineering: a safe server proxy for an external API, deterministic procedural generation, a 3D scene that has to stay fast, and an accessible fallback for people who can't use WebGL.

How a repository becomes a grave

Status	Days since last push	What you see
Alive	0 to 30	Clean pale stone, green glow, sprouts, no fog patch
Forgotten	31 to 90	Slightly worn stone, slight tilt, an amber lantern
Buried	91 to 365	Darker, mossy, tilted, more fog
Ancient	More than 365	Cracked, heavily tilted, ivy, thick fog, a faint purple glow
Archived	Any, if archived	Pale upright memorial with a wreath and soft moonlight

Stars	Marker	Shape
0 to 4	Gravestone	Simple slabs in three silhouettes
5 to 19	Decorative	Gothic and ornate slabs, one star
20 to 99	Elaborate	Crosses, winged and ringed memorials, two stars
100 or more	Monument	Obelisk, column or small temple on a plinth, three stars

Status is an approximate activity signal from `pushed_at`, not a judgement of code quality. The interface says so, because a finished, stable project can look ancient.

© 2026 Gopinath. All rights reserved.

Section 02

Every feature

Area	What it does
Landing	Form carved into a headstone, instant username validation, link to the demo. The 3D code starts downloading when the input gets focus.
Real data	Public repositories fetched through a server proxy, up to 500 per account, with an optional token.
Demo	`/demo` shows invented repositories, clearly labelled, with dates relative to today so that every status appears.
Intro	A 7-second camera move through opening gates. Skip it with the button, Esc or Space.
Exploring	Orbit, zoom, hover to highlight, click to focus. The camera stays above ground and inside the cemetery.
Details panel	Status, epitaph, description, language, stars, forks, marker, dates and a GitHub link.
Find and filter	Search and status filters dim non-matching graves, so the layout never moves. A forks toggle changes the collection.
Demo Resurrection	A 3.5-second effect that repairs a grave for the current visit. It changes nothing on GitHub, and the interface says so.
Camera controls	Reset camera returns to the overview, Home view returns to the gate side, and the top-right button returns to the landing page.
Quality	High and low levels, chosen automatically and switchable by hand. Falls back to low if frame rate suffers.
Accessibility	A list view gives every grave a keyboard path, Esc closes the panel, reduced motion is respected, and the lightning is one flash that never strobes.
Fallback	If WebGL is missing, the context is lost or the scene crashes, the app shows the full list instead of a blank page.
Clear states	Separate messages for loading, not found, rate limited (with a countdown), GitHub failure, empty account and forks only.
Sharing	Each graveyard has its own URL, a copy link button, page metadata and a social preview image.

© 2026 Gopinath. All rights reserved.

Section 03

How it looks

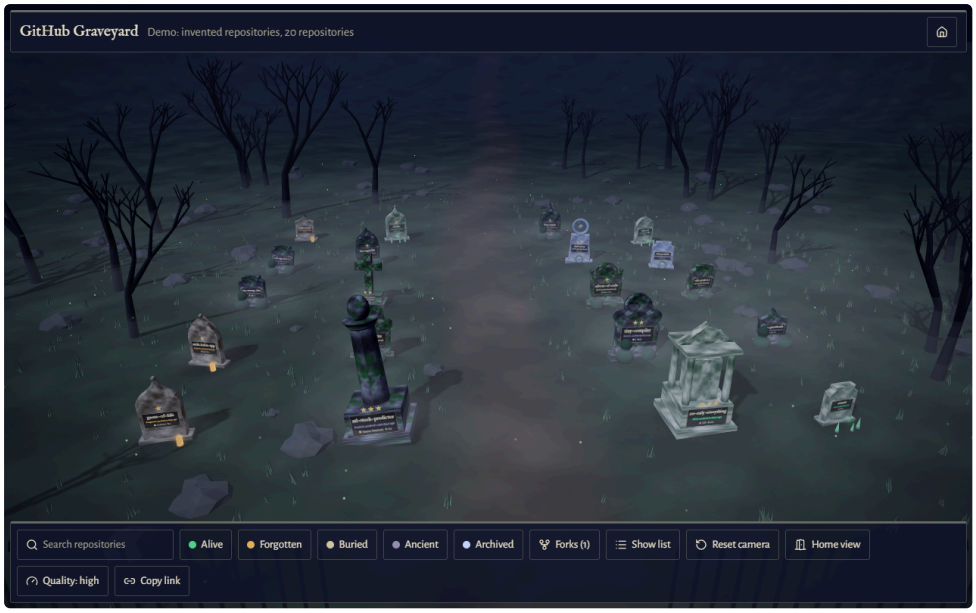


Landing page: the form is carved into a headstone.



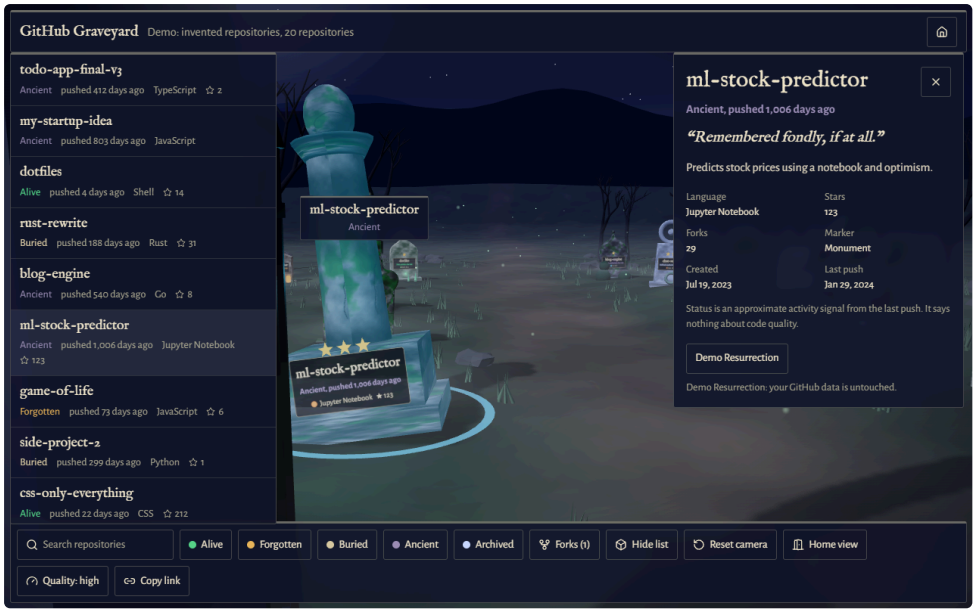
Intro: the view through the gate before the camera moves in.

© 2026 Gopinath. All rights reserved.



Overview of the demo cemetery, with the search, filter and camera controls below.

© 2026 Gopinath. All rights reserved.



A selected grave with the list and details panel open.



The 100-grave cap, using mocked repositories. Monuments take the slots nearest the path.

Screenshots come from automated test runs that use software rendering, so a real graphics card may look slightly different.

© 2026 Gopinath. All rights reserved.

Section 04

Tech stack

Technology	Version	Used for	Why this one
Next.js	16.4.3	Pages and the API route	One project holds both the UI and the server code that must keep the token private.
React	19.3.0	UI components	Required by Next.js, and React Three Fiber builds on it.
TypeScript	5.9.3	All code, in strict mode	`noUncheckedIndexedAccess` forces array lookups to handle missing items.
Three.js	0.187.0	3D rendering	The standard WebGL library for the web.
React Three Fiber	9.8.2	Scene as React components	Lets the scene share state and lifecycle with the UI.
Drei	10.7.9	Orbit controls, HTML labels, adaptive quality	Tested helpers instead of hand-written ones.
GSAP	3.15.0	Intro, camera and resurrection timelines	Precise timelines that can be killed cleanly.
Zustand	5.0.16	Shared UI state	Selectors stop a hover from rerendering every grave.
Tailwind CSS	4.3.3	Interface styling	Theme colours live in one place.
Lucide	1.66.0	Icons	Small, consistent icon set.
Vitest	5.0.5	Unit and API tests	Fast, and needs no browser.
Playwright	1.64.1	Browser tests, screenshots	Runs a real production build in Chromium.
ESLint	9.39.5	Code checks	The Next.js rule set, with frame-mutation rules relaxed only for scene code.

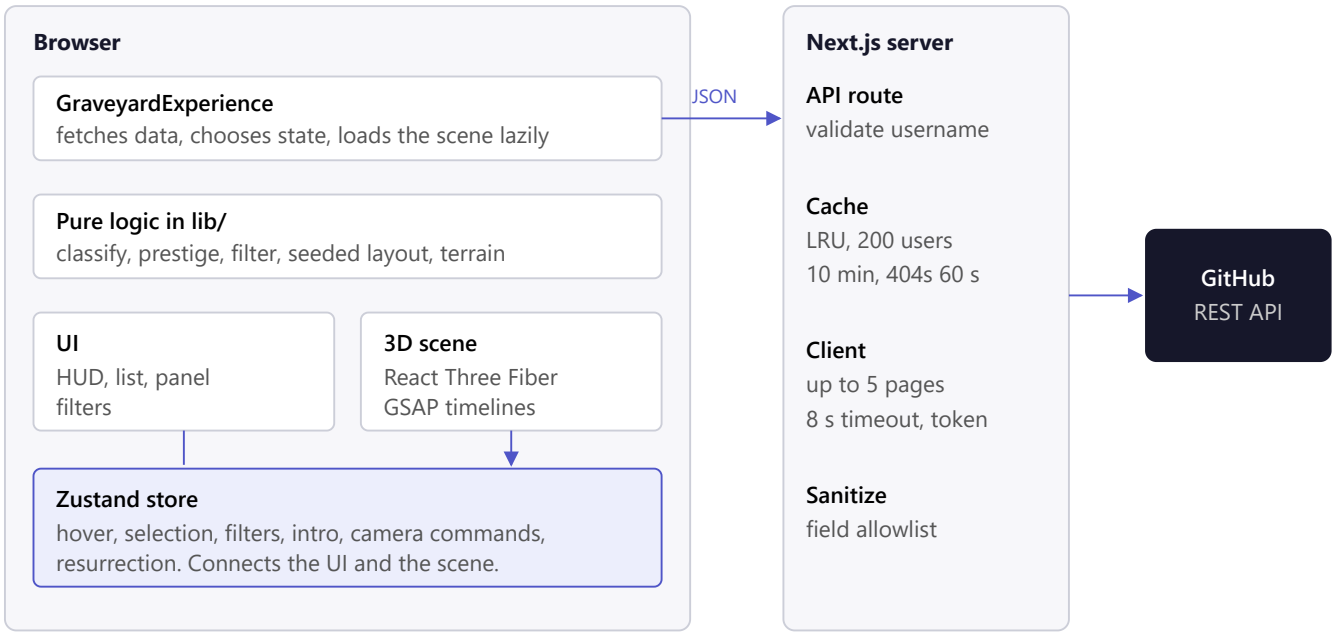
Fonts are IM Fell English for display text and Alegreya Sans for the interface. The external service is the GitHub REST API. There is no database or paid service.

© 2026 Gopinath. All rights reserved.

Section 05

Architecture

The browser never talks to GitHub directly. A server route validates the request, calls GitHub, strips the response to known fields and caches it. Everything else, including classification and layout, runs in the browser from that clean data.



Three rules behind the design

Secrets stay on the server	The token is read only in server code, and the URL is built only from a validated username, so callers can't reach other GitHub paths.
Logic is separate from drawing	Rules live in pure functions with an explicit date or seed, so they can be tested without a browser.
Fast values skip React	Per-frame animation changes Three.js objects directly. React state only changes for real events, such as a hover or selection.

© 2026 Gopinath. All rights reserved.

Section 06

How each part works

Proxy	`/api/github/[username]` validates the name, then returns data or one of five error codes: invalid username 400, not found 404, rate limited 429, upstream error 502, timeout 504.
Cache	An in-memory least-recently-used map. Simultaneous requests for one user share a single GitHub call, and an expired copy is served if GitHub fails. Forks are filtered after the cache, so toggling them spends no quota.
Classification	Counts whole UTC calendar days, so status doesn't shift with the time of day. It runs on the client, so cached data never shows an old status. A missing push date falls back to the creation date, and the panel says so.
Layout	FNV-1a hashes the username and sorted repository IDs into a seed for a mulberry32 random generator. The same input gives the same cemetery, in any order. Graves sit in grid cells beside the path, and their jitter is smaller than the cell margin, so overlap is impossible by construction.
Terrain	`terrainHeight(x, z)` is one pure function shared by the ground mesh, grave placement and the camera clamp, so all three always agree.
Graves	Twelve shared geometries, three per marker tier. Hover and selection swap between shared materials rather than cloning one per grave.
Labels	All labels are drawn once onto one canvas texture, and each grave's plane points at its own cell. The atlas has exactly 100 cells, matching the render cap.
Instancing	Trees, rocks, grass, fence bars, shadows, stars and grave accessories are instanced. Status glows are a single point cloud.
Camera	`CameraDirector` owns one GSAP timeline and kills it before any new move. Each frame it also keeps the camera above the terrain and its target inside the cemetery.
Resurrection	One timeline: dim, lightning, cracks, ghost, repair, sparks, finish. State is kept only in memory and is killed safely if you leave partway.
Fallback	A WebGL check, a React error boundary and a context-lost listener all lead to the same list view.

© 2026 Gopinath. All rights reserved.

Section 07

Key decisions

The full reasoning is kept in `decisions.md` in the repository.

Decision	Why
Server proxy, not browser calls	Keeps the token private, allows caching across visitors and limits what reaches the browser.
Own cache, not the Next data cache	Allows serving stale data on rate limits, shorter caching for 404s, and direct testing.
`/demo` **, not** `/graveyard/demo`	`demo` could be a real GitHub username.
Cap of 100 graves in 3D	Keeps rendering within budget. The list still shows every fetched repository, and a notice explains the cap.
Dim filtered graves, don't move them	A stable layout keeps people oriented while they search.
Label atlas, not text meshes	Individual text would need about 300 extra meshes at the cap.
Own seeded random	About 15 lines give repeatable results, with no dependency.
No post-processing	Glow comes from additive sprites and emissive materials, avoiding a dependency and a full-screen GPU pass on each frame.
SVG landing page, not WebGL	The first screen appears immediately, and the 3D code loads only when needed.
`cacheComponents` **turned off**	With it on, Next keeps earlier routes alive in a hidden Activity, which would keep a heavy WebGL context around after leaving.
Gate cutaway	The gate hides when the camera is above 8 units, so it doesn't block the overview. Moving the camera would have cut off the front rows.

© 2026 Gopinath. All rights reserved.

Section 08

Repository map

About 4,600 lines of TypeScript across the app, components, lib and tests.

Path	Purpose
app/page.tsx	Landing page
app/demo/	Demo graveyard
app/graveyard/[username]/	Validates the username, sets metadata, renders the experience
app/api/github/[username]/	Server proxy route
app/opengraph-image.tsx	Generated social preview image
lib/github/	GitHub client, cache, sanitizing, errors
lib/classify, prestige	Status and marker rules
lib/seed, layout, terrain	Procedural generation
lib/filter, epitaphs, languages	Search, render cap, epitaph text, language marks
lib/store, quality, timelines	UI state, quality settings, animation timing
lib/username, webgl, demo-data	Validation, WebGL check, invented demo repositories
components/ui/	Landing form, HUD, filters, list, panel, state messages, `GraveyardExperience`
components/three/	Canvas, camera, sky, terrain, fog, gate, scenery, particles, tombstones, stars, glows, resurrection
tests/unit/	Domain logic and API route tests
tests/e2e/	Browser flows, draw call budgets, review screenshots
decisions.md	Running log of what changed and why

© 2026 Gopinath. All rights reserved.

Section 09

Install, run and deploy

Run from source

```
git clone https://github.com/goprocker/GitHubGraveyard.git
cd GitHubGraveyard
npm install
npm run dev
```

Open `http://localhost:3000`. Developed and tested on Node 24.

Environment variables

Variable	Purpose
GITHUB_TOKEN	Optional. Raises the GitHub limit from 60 to 5,000 requests an hour. Read only on the server, and needs no scopes for public data.
NEXT_PUBLIC_E2E	Tests only. Set by the Playwright config. Leave it unset in production.

Deploy

Vercel: import the repository and optionally set `GITHUB_TOKEN`.

Any Node host:

```
npm ci
npm run build
GITHUB_TOKEN=your_token npm start
```

These deployment steps haven't been run on a hosted service yet. Only the local production build has been tested.

© 2026 Gopinath. All rights reserved.

Section 10

Testing and verified results

```
npm run lint        npm run typecheck
npm test            npm run test:e2e
```

Check	Result on 31 Oct 2026
Lint and typecheck	Clean
Unit and API tests	52 of 52 pass
Browser tests	11 of 11 pass on a production build, with Chromium and software rendering
Draw calls at 100 graves	339 on high (budget 450), 225 on low (budget 250)
Live GitHub	`octocat` rendered in the scene with no page errors, and a nonexistent username returned 404

© 2026 Gopinath. All rights reserved.

Section 11

Limits and known gaps

Public data only	Public repositories, at most 500 per account. A full last page is reported as truncated.
Render cap	At most 100 graves in 3D, chosen by stars, then most recent push, then ID.
Cache scope	Per server instance. It resets on restart and isn't shared between instances.
No per-IP limit	All visitors share the server's GitHub quota.
Resurrection	Visual only, for the current visit. It doesn't detect real revivals. Its ghost and crack visuals haven't been confirmed by eye: tests prove only that the sequence completes.
Frame rate	Not measured on a real GPU, because the automated tests use software rendering.
Dependencies	`npm audit` reports 16 moderate warnings that haven't been reviewed.
Small visual trade-offs	The gate pops out of view instead of fading, stars skip the small hover lift, and stars snap upright on repair.

© 2026 Gopinath. All rights reserved.

Words and data flow

Term	What it means here
Draw call	One instruction to the GPU to draw a batch. Fewer calls usually means smoother frames.
Instancing	Drawing many copies of one shape in a single call, each with its own position.
Texture atlas	One image holding many smaller ones. Here, every grave label.
Procedural generation	Building content from rules and a seed instead of by hand.
Deterministic	The same input always gives the same output.
LRU cache	When full, it drops the entry unused for the longest time.
Proxy	A server that makes requests on the browser's behalf.
Allowlist	Copying only named fields, so anything unexpected is dropped.
Selector	A function picking one value from the store. A component rerenders only when that value changes.
Code splitting	Downloading heavy code only when needed. Here, the 3D scene.

Follow one request

1. You type a username. usernameProblem() checks it, then the app opens /graveyard/name.
2. The server page validates it again and renders GraveyardExperience.
3. The browser calls /api/github/name.
4. The route checks the cache. On a miss, it fetches up to 5 pages from GitHub.
5. sanitizeRepo() keeps known fields only, and the result is cached.
6. The browser computes each repository's status and marker with one "now".
7. selectRendered() picks up to 100, and generateLayout() places them from the seed.
8. The 3D chunk loads, and the intro plays.
9. Clicking a grave calls store.select(id). The camera moves and the panel opens.

© 2026 Gopinath. All rights reserved.

Section 13

Hard problems and what they taught

Problem	Cause	Fix	Lesson
"Skip intro" sometimes did nothing	The button appears before the lazy 3D code loads, so early clicks had no listener.	The scene checks for an earlier skip request when it mounts.	Lazy loading creates timing gaps. Store the request, not just the event.
Over the draw call budget: 517 and 403	Up to three separate star meshes per grave, and separate static gate parts.	Instanced the stars and merged the gate, reaching 338 and 224.	Measure at the worst case. The 20-repo demo hid this completely.
Giant arcs in the sky, with a spike through them	The ground rim rose without limit, and path flattening cut a notch through it.	Capped the rim, and faded the flattening out across it.	Passing tests can't judge visuals, so review real screenshots.
Page colours were missing	`@theme inline` emits no CSS variables, but plain CSS referred to them.	Switched to `@theme`.	Know what your tooling actually generates.
Shadows would have rendered as squares	An alpha map reads the green channel, which stayed full on a transparent canvas.	Painted the gradient on opaque black.	Check what a texture channel really contains.
White area in a test screenshot	Software-rendering paint glitch in the test setup, not an app bug.	Confirmed with a run without those flags. No app change was needed.	Isolate the variable before changing code.

© 2026 Gopinath. All rights reserved.

Section 14

Interview questions and cheat sheet

Understand these, then say them in your own words.

Your 20-second pitch
GitHub Graveyard turns a GitHub profile into a 3D cemetery. A Next.js route safely proxies and caches GitHub data, the browser generates a repeatable cemetery from it, and Three.js renders up to 100 graves within a draw call budget, with a keyboard-friendly list as the fallback.

Why not call GitHub from the browser?
The token would be exposed, each visitor would have their own low rate limit, and there would be no shared cache. The proxy also strips the response to known fields.

How is the layout repeatable?
IDs are sorted and hashed with the username into a seed for my own random generator. There's no `Math.random` in placement, so input order doesn't matter.

How do you guarantee graves never overlap?
By construction. Each grave has its own grid cell, and jitter is limited to the cell margin. A unit test confirms it at 0, 1, 5, 50 and 100 graves.

How did you keep it fast?
Shared geometry and materials, instancing, one label atlas, a render cap, and a measured budget that a test enforces. Hover uses selectors, so at most two graves rerender. I haven't measured frame rate on a real GPU yet.

What happens when GitHub rate limits you?
The route returns 429 with Retry-After, or serves an expired cached copy if it has one. The UI shows a countdown before retry and points to the demo.

How is it accessible?
Every 3D action has a list-based keyboard path. Reduced motion shortens the intro and removes the flash. Without WebGL, the list becomes the app.

What would you do next?
Measure frame rate on real GPUs, deploy it, add per-IP rate limiting, and move the cache to a shared store if traffic justifies it.

Numbers to remember

Fetch limit	5 pages, 500 repos, 8 s timeout	Cache	200 users, 10 min, 404s 60 s
Render cap	100 graves	Draw calls	339 high, 225 low
Status edges	30, 90, 365 days	Star edges	5, 20, 100
Intro	7 s	Resurrection	3.5 s
Tests	52 unit, 11 browser	GitHub limit	60 an hour, 5,000 with a token

© 2026 Gopinath. All rights reserved.